

COL774 A4 Report

Transforming Hand-Written Equations to LaTeX

Generated	Actual
$\frac{1}{2}(\phi') = -\frac{1}{2}\phi^2$	$\frac{1}{2}\phi(-\lambda)\phi = -\frac{\lambda}{2}\phi^2$

Samarth - 2021MT60942

Atul - 2021MT10623

28.11.2023

Tuesday

INTRODUCTION

We were presented with the problem of converting handwritten formulae to latex code. This problem is important for students and researchers, as they may want to convert their handwritten works to LaTeX reports, and ML models can make the process a lot faster.

MODEL

- Deployed an Encoder-Decoder architecture
- Encoder contains convolutional layers, activated by ReLU, and max-pooled
- Decoder contains an LSTM layer, combined with a learnable embedding layer and a linear output layer.

DETAILED COMPONENTS OF PROGRAM

- **Vocabulary CLASS:** To move from index space to word space and vice-versa.
- **LatexFormulaDataset CLASS:** When initialised, we first load the given CSV files and convert them to pandas dataframes. Then, we add <start>, <end> and <unk> tokens at end of each formula. When dataset[index] is called, the formula is converted to index tokens, and image is transformed.
- **TestingDataset CLASS:** Handles the handwritten dataset separately. It has a different transform and loading procedure, as formulas are unavailable.
- **EncoderCNN CLASS:** Implements the encoder as described in assignment statement
- **DecoderLSTM CLASS:** Implements the decoder as described in assignment statement. Takes input context and target tensor (during training for teacher forcing), and outputs the predicted formula.
- **HandwritingToLatexModel CLASS:** Combines the encoder and decoder as a single model. Facilitates training together.

TRAINING

1. ON SYNTHETIC DATA
 - Method of teacher-forcing was used to improve training efficiency.
 - We used a random number generator to decide when to use teacher forcing.
 - The flow of training is as follows. First, the CNNEncoder produces a context vector after encoding the image through its layers. Now, we pass the context to the decoder. At each timestep, the LSTMDecoder gets context concatenated with either previous timestep output or previous timestep ground truth token depending on teacher-forcing. The outputs of the LSTM are argmaxed to create the output formula

2. ON HANDWRITTEN DATA

- Here we have employed a transfer learning approach. First we train the model on large synthetic data, and it learns to decode mathematical text, which is perfectly formatted.
- We continue the training on handwritten data. This time the model learns to transform wiggly text into latex code. The Encoder become intelligent.
- This training is much faster as compared to the original synthetic data training, as seen in Table 4.1.

RESULTS

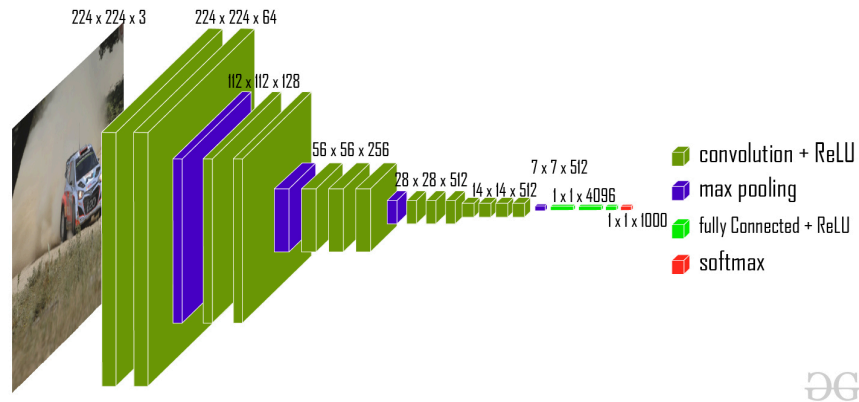
Model	Dataset	BLEU Score
No Fine Tuning	Synthetic: Validation	0.22753
	Synthetic: Test	0.23002
	Handwritten: Validation	0.12236
Fine Tuned On Handwritten Training Set	Synthetic: Validation	0.08518
	Synthetic: Test	0.08704
	Handwritten: Validation	0.29899

COMPETITIVE MODELS

We suspected that our basic CNN encoder

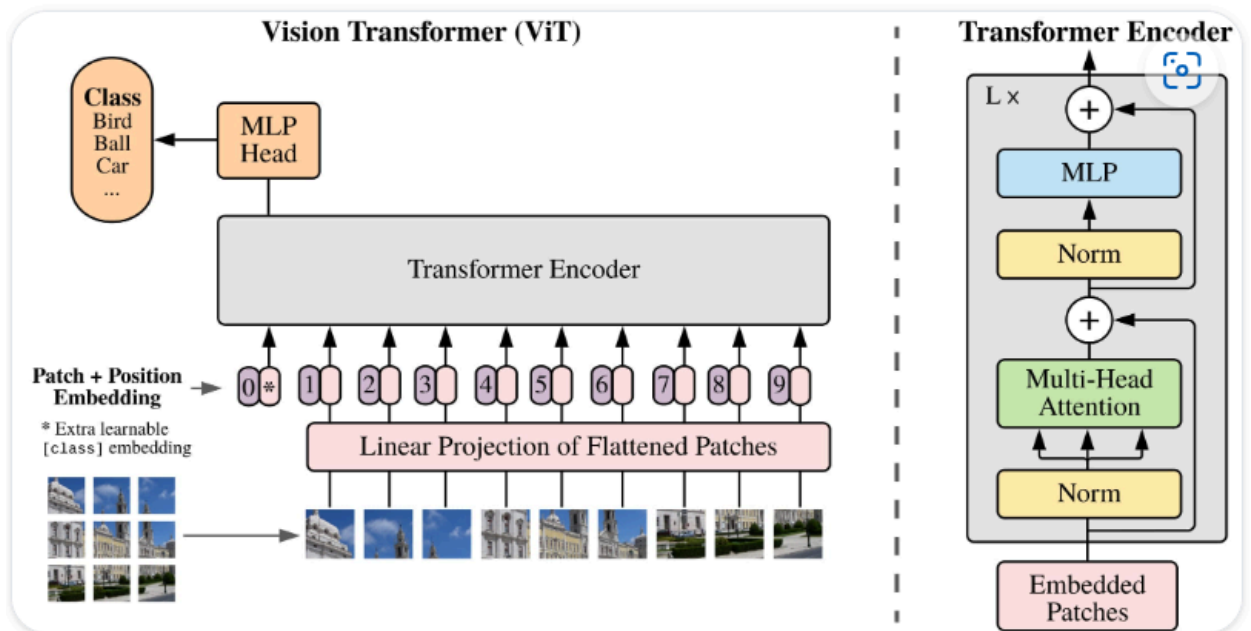
- VGG16 as Encoder: We used pretrained weights of VGG model as encoder. The VGG is a 16-layer deep convolutional network, originally designed for image classification and object detection in 2014 ILSVRC competition. These were obtained directly from PyTorch.

- The Classification Layer at end of VGG has 3 Linear layers. Since the task of classification is different from our task, we replaced the last linear layer with desired output size of 512. Other layers were frozen during training.



(VGG Architecture)

- ViTModel as Encoder: Here, we used ViTModel from huggingface and trained for 3 epochs. Then followed by 15 epochs of fine-tuning on handwritten data. It gave 0.108 BLEU on leaderboard. The issue was very slow training and prediction due to a dense model.



Challenges faced and interesting observations

0. Training duration and epochs used; On the main synthetic data,
1. The initial training period was a bit confusing. The model showed low loss, but upon printing we found that it is printing only <PAD> tokens. We improved this by ignoring the <PAD> token from the loss backpropagation.
2. While debugging the code, we accidentally mismanaged the teacher-forcing method. At each position, we passed the expected output of that position. The method reduced the training error drastically and made us happy. But we realised that the model just learned to decode the embedding it was given as input. And in testing there are no teacher forcing labels, so the model will fail.
3. On the Handwritten test dataset, many of the outputs were coming out to be unknowns due to mismatch in vocabulary. So we replaced the <UNK> token, with the second highest probability token. The score improved a little bit.